

LoRA

📅 date	2026년 9월 10일
# Num	11
🔗 논문 원문 링크	https://arxiv.org/pdf/2106.09685
🔗 발제자료	https://www.canva.com/design/DAHUOCe_3qY/2Dzy1z-OO0QLsVvY1kjTPg/edit?ui=eyJBIjp7fX0
≡ 유형	BOAZ 학기 2주차 (1) - 발제

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Background

Rank

Full fine-tuning vs LoRA — 학습 대상

파라미터 개수 계산

Abstract

1. Introduction

LoRA의 장점

용어 정의

2. Problem Statement

3. AREN'T EXISTING SOLUTIONS GOOD ENOUGH?

4. Our Method

4.1 LOW-RANK-PARAMETRIZED UPDATE MATRICES

4.2 APPLYING LORA TO TRANSFORMER

5. Empirical Experiments

5.1 Baselines

1. Fine-Tuning (FT)

2. Bias-only / BitFit

3. Prefix-embedding tuning (PreEmbed)

4. Prefix-layer tuning (PreLayer)

5. Adapter tuning

6. LoRA

5.2 RoBERTa - Base/Large

5.3 DeBERTa XXL

5.4 GPT-2 - Medium/Large

5.5 Scaling up to GPT-3 175B

6. Related Works

6.1 Transformer Language Models

6.2 Prompt Engineering and Fine-Tuning

6.3 Parameter-Efficient Adaptation

6.4 Low-Rank Structures in Deep Learning

7. UNDERSTANDING THE LOW-RANK UPDATES

7.1 WHICH WEIGHT MATRICES IN TRANSFORMER SHOULD WE APPLY LORA TO?

7.2 WHAT IS THE OPTIMAL RANK r FOR LORA?

7.2.1 Subspace similarity between different r

7.2.2 Subspace similarity between different random seeds

7.3 HOW DOES THE ADAPTATION MATRIX ΔW COMPARE TO W ?

8. CONCLUSION AND FUTURE WORK

Background

Rank

- 행렬의 rank: "그 행렬이 표현할 수 있는 독립적인 정보의 차원 수"
- $d \times d$ 크기의 정방행렬이면 최대 rank d 까지 가능(=full rank)
- 그보다 작은 rank r 을 가진 행렬은 $d \times r$ 행렬과 $r \times d$ 행렬의 곱으로 압축 표현 가능
→ 원래는 $d \times d = d^2$ 개의 숫자로 표현되던 행렬을, rank가 r 이라면 $d \times r + r \times d = 2dr$ 개의 숫자만으로도 정확히 표현
- $r \ll d$ 이면 $2dr \ll d^2$ 이니 훨씬 적은 파라미터로 같은 정보를 나타낼 수 있음

Full fine-tuning vs LoRA — 학습 대상

- Full fine-tuning: pretrained weight W ($d \times d$) 자체를 통째로 업데이트
- 학습 파라미터 수: d^2 개
- LoRA: W 는 frozen
- 변화량 ΔW 를 BA로 저랭크 근사
- 실제 학습 대상은 B 와 A 뿐
- dense layer 전체(W)는 안 건드리고, 위에 붙는 작은 보조 경로(BA)만 학습
- W 자체의 일부 요소를 골라 학습하는 게 아니라, W 는 통째로 얼리고 별개의 작은 행렬 쌍을 새로 만들어 학습하는 구조

파라미터 개수 계산

- $W \in R^{d \times d}$ 라고 할 때
- $B \in R^{d \times r}$ - 파라미터 $d \times r$ 개
- $A \in R^{r \times d}$ - 파라미터 $r \times d$ 개

- 합산 학습 파라미터: $2dr$ 개
- 예시: GPT-3의 $d_{model} = 12288, r = 1$ or 2인 경우
- $r = 1$ 일 때: $2 \times 12288 \times 1 = 24,576$ 개
- Full fine-tuning이라면: $12288^2 \approx 1.5$ 억 개
- 논문에서 말하는 "10,000배 감소" 확인

Abstract

자연어 처리의 중요한 패러다임은 일반 도메인 데이터에 대한 대규모 사전 학습과 특정 작업 또는 도메인에 대한 적응으로 구성됩니다. 더 큰 모델을 사전 학습함에 따라 모든 모델 파라미터를 재학습하는 전체 파인튜닝(full fine-tuning)은 점점 실행하기 어려워지고 있습니다. GPT-3 175B를 예로 들면, 각각 175B 파라미터를 가진 파인튜닝된 모델의 독립적인 인스턴스를 배포하는 것은 비용이 너무 많이 듭니다. 우리는 사전 학습된 모델 가중치를 고정하고 Transformer 아키텍처의 각 레이어에 학습 가능한 랭크 분해 행렬(rank decomposition matrices)을 주입하여 다운스트림 작업을 위한 학습 가능한 파라미터 수를 크게 줄이는 Low-Rank Adaptation, 즉 LoRA를 제안합니다. Adam으로 파인튜닝된 GPT-3 175B와 비교했을 때, LoRA는 학습 가능한 파라미터 수를 10,000배, GPU 메모리 요구량을 3배 줄일 수 있습니다. LoRA는 더 적은 학습 가능한 파라미터, 더 높은 학습 처리량(throughput)을 가지며, 어댑터(adapter)와 달리 추가적인 추론 지연 시간(inference latency)이 없음에도 불구하고 RoBERTa, DeBERTa, GPT-2, GPT-3에서 파인튜닝과 동등하거나 더 나은 모델 품질을 보여줍니다. 또한 우리는 언어 모델 적용에서의 랭크 부족(rank-deficiency)에 대한 실증적 조사를 제공하며, 이는 LoRA의 효능을 밝혀줍니다. 우리는 LoRA와 PyTorch 모델의 통합을 용이하게 하는 패키지를 공개하며, RoBERTa, DeBERTa, GPT-2에 대한 구현 및 모델 체크포인트를 <https://github.com/microsoft/LoRA> 에서 제공합니다.

1. Introduction

1) 배경: 왜 이 문제가 중요해졌는가

- NLP의 많은 응용은 하나의 대규모 사전학습 언어모델을 여러 다운스트림 태스크에 적응시키는 데 의존
이 적응은 일반적으로 사전학습 모델의 모든 파라미터를 업데이트하는 파인튜닝으로 이루어짐
파인튜닝의 주요 단점 → 새로운 모델이 원본 모델과 동일한 수의 파라미터를 가짐
- 몇 달마다 더 큰 모델이 학습되면서 이 문제의 성격이 변화
GPT-2, RoBERTa large 시절엔 단순한 "불편함" 수준이었으나, 1,750억 개 파라미터를 가진 GPT-3에 이르러선 심각한 배포 과제로 격상됨

2) 기존 완화 시도와 그 한계

- 많은 연구자가 일부 파라미터만 적응시키거나 새로운 작업용 외부 모듈을 학습하는 방식으로 이를 완화하려 시도
이 방식의 장점 → 태스크마다 사전학습 모델 외에 소수의 작업별 파라미터만 저장/로드하면 되므로 배포 시 운영 효율성 크게 향상

그러나 기존 기술들의 두 가지 문제 (Section 3에서 다룸)

- 모델 깊이를 확장해 추론 지연(inference latency)을 유발 (Houlsby et al., 2019; Rebuffi et al., 2017)
- 모델이 사용 가능한 시퀀스 길이를 감소시킴 (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021)

더 중요한 문제 → 이런 방법들이 종종 파인튜닝 베이스라인 성능에 미치지 못해, **효율성 vs 모델 품질** 사이의 트레이드오프 발생

3) LoRA의 아이디어가 나온 배경

영감의 출처 → Li et al. (2018a), Aghajanyan et al. (2020)

이 연구들은 학습된 과매개변수화(over-parametrized) 모델이 사실상 낮은 고유 차원(intrinsic dimension)에 존재함을 보임

저자들의 가설 → 모델 적응 과정에서의 가중치 변화 역시 낮은 "고유 랭크(intrinsic rank)"를 가질 것

이 가설을 바탕으로 제안된 방법론이 Low-Rank Adaptation(LoRA)

- LoRA를 사용하면 신경망의 일부 밀집 층(dense layer)을 간접적으로 학습시킬 수 있는데, 이는 적응 과정에서 발생하는 밀집 층의 변화를 랭크 분해 행렬(rank decomposition matrices)로 최적화하는 동시에 사전 학습된 가중치를 고정(frozen) 상태로 유지함으로써 가능하며, 그림 1에 나타나 있습니다.

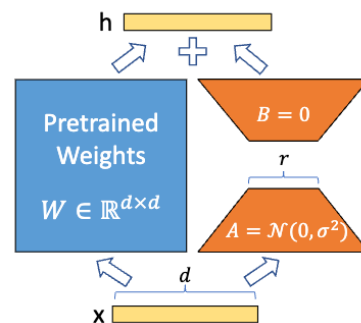


Figure 1: Our reparametrization. We only train A and B .

- 왼쪽 파란 상자 (Pretrained Weights W): 입력 x 가 그대로 W 를 통과하는 경로 - frozen, 학습 안 됨
- 오른쪽 주황색 경로 ($A \rightarrow B$): 같은 입력 x 가 A 를 거치고 B 를 거치는 경로 - 이 부분만 학습
- 두 경로의 출력을 더해 h 산출: $h = Wx + BAx$
- A 가 아래쪽(입력 쪽), B 가 위쪽(출력 쪽)에 위치한 이유
- 순서: $x \rightarrow A \rightarrow (\text{중간차원 } r) \rightarrow B \rightarrow \text{출력}$

$A \sim \mathcal{N}(0, \sigma^2)$ 과 $B = 0$ 의 의미

- $A \sim \mathcal{N}(0, \sigma^2)$: A 행렬의 각 원소를 평균 0, 분산 σ^2 인 정규분포(가우시안 분포)에서 무작위 추출해 초기화
 - 일반적인 신경망 가중치 초기화(random init) 방식과 동일
 - $B = 0$: B 행렬의 모든 원소를 0으로 초기화
 - 조합 결과: 학습 시작 시점에 $\Delta W = BA = 0$
 - 이유: B 가 전부 0이므로 A 가 어떤 값이든 곱하면 0
 - 학습 시작 순간엔 LoRA 추가 이전과 동일한 모델로 출발
 - 이후 학습 진행되며 B 와 A 가 서서히 업데이트, ΔW 가 의미 있는 값 확보
→ 안정적인 학습 시작을 보장하는 장치
 - 반대로 B 를 랜덤 초기화했다면 학습 시작부터 pretrained 모델 출력이 무작위로 흔들려 불안정해짐
- GPT-3 175B를 예로 들면, 전체 랭크(즉, d)가 12,288로 매우 높더라도 매우 낮은 랭크(즉, 그림 1의 r)은 1 또는 2가 될 수 있음만으로도 충분하며, 이를 통해 LoRA는 저장 공간과 연산 효율성을 모두 확보

LoRA의 장점

- 사전학습 모델을 공유하며 다양한 작업용 소형 LoRA 모듈을 여러 개 구축 가능
- 공유 모델은 고정하고 Figure 1의 행렬 A, B만 교체 → 효율적인 작업 전환, 저장 공간 요구사항과 작업 전환 오버헤드 대폭 절감
- 학습 효율성 향상 → 적응형 옵티마이저(Adam 등) 사용 시 대부분의 파라미터에 대한 gradient 계산/옵티마이저 상태 유지 불필요 → 하드웨어 진입 장벽 최대 3배 절감
- 주입된 훨씬 작은 저랭크 행렬만 최적화하면 됨
- 단순한 선형 설계 덕분에 배포 시 학습된 행렬을 고정 가중치와 병합 가능 → 구조적으로 완전 파인튜닝 모델 대비 추론 지연 전혀 유발 안 함
- 기존의 많은 방법들과 직교(orthogonal)적 관계 → prefix-tuning 등 다른 방법과 결합 가능 (Appendix E에 예시 제공)

용어 정의

Transformer 아키텍처에 대해 관례적인 용어를 사용

- Transformer 층의 입력 및 출력 차원 크기: d_{model}
- 셀프 어텐션(self-attention) 모듈의 query/key/value/output projection matrices을 지칭하기 위해 W_q, W_k, W_v, W_o 를 사용
- W 또는 W_o : 사전 학습된 가중치 행렬
- ΔW 는 적응 과정에서의 누적 그래디언트 업데이트
- LoRA 모듈의 랭크를 나타내기 위해 r 을 사용합니다.
- 저희는 (Vaswani et al., 2017; Brown et al., 2020)에서 정한 관례를 따르며, 모델 최적화를 위해 Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017)을 사용하고, Transformer MLP 피드포워드 차원을 $d_{ffn} = 4 \times d_{model}$ 로 사용

2. Problem Statement

1) 기본 세팅

사전학습된 언어모델이 하나 있다고 가정 → 이 모델은 $P_\Phi(y|x)$ 로 표현됨

- x : 입력 (예: 질문, 기사 원문)
- y : 출력 (예: 답변, 요약문)
- Φ : 모델의 파라미터 전체 (예: GPT-2, GPT-3의 모든 가중치)

즉 "파라미터 Φ 를 가진 모델이 x 를 보고 y 를 생성할 확률"이라는 뜻논문은 NL2SQL(자연어 → SQL 변환), 요약, MRC(기계독해) 같은 다운스트림 태스크를 예시로 들

2) 학습 데이터 표현

각 다운스트림 태스크는 (x_i, y_i) 쌍으로 이루어진 데이터셋 $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$ 으로 주어짐
→ 예를 들어 NL2SQL이면 x_i 는 자연어 질문, y_i 는 그에 대응하는 SQL 쿼리

3) Full fine-tuning의 목적함수

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t}))$$

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad (1)$$

- pretrained weight $W \in \mathbb{R}^{d \times d}$ 자체를 통째로 업데이트
- 학습해야 할 파라미터 수 = d^2 개
- 정답 시퀀스 y 를 한 토크씩(y_t) 생성할 때, 그 앞의 문맥(x 와 이미 생성한 $y_{<t}$)이 주어졌을 때 정답 토크를 낼 확률의 로그값을 최대화하자는 뜻
 - 이는 일반적인 언어모델의 학습 목적함수(다음 토크 예측)와 동일
 - 다만 여기서는 사전학습된 파라미터 Φ_0 에서 출발해, 이를 업데이트하며 목적함수를 최대화 $\rightarrow \Phi_0 \rightarrow \Phi_0 + \Delta\Phi$ 로 파라미터 전체가 변경됨

4) Full fine-tuning의 문제점

- 태스크마다 학습해야 하는 $\Delta\Phi$ 의 크기가 원래 모델 크기 $|\Phi_0|$ 와 동일하다는 게 핵심 문제
- GPT-3처럼 $|\Phi_0| \approx 1750$ 억 개면, 태스크 하나를 배포할 때마다 1750억 개짜리 파라미터 세트를 통째로 저장/배포해야 함
태스크가 10개, 100개로 늘어나면 감당 불가능한 수준이 됨

5) LoRA(파라미터 효율적 접근)의 목적함수

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t}))$$

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (2)$$

- Eq. 1과 형태는 거의 동일하나, 딱 하나 다른 점 존재
 $\Delta\Phi$ 를 직접 최적화하는 대신,
 - 1) $\Delta\Phi$ 를 훨씬 작은 파라미터 집합 Θ 의 함수로 표현($\Delta\Phi = \Delta\Phi(\Theta)$)
 - 2) 그 Θ 만 최적화
 이때 $|\Theta| \ll |\Phi_0| \rightarrow \Theta$ 는 원래 모델 크기보다 훨씬 작음
- Full fine-tuning: $\Delta\Phi$ 자체가 학습 대상, 크기 = $|\Phi_0|$
- LoRA(제안 방법): $\Delta\Phi$ 를 만들어내는 더 작은 파라미터 Θ 가 학습 대상, 크기 = $|\Theta| \ll |\Phi_0|$

논문은 마지막에 GPT-3 175B에서는 이 $|\Theta|$ 가 $|\Phi_0|$ 의 0.01%밖에 안 될 수 있다고 언급

3. AREN'T EXISTING SOLUTIONS GOOD ENOUGH?

1) 이 섹션의 목적

지금까지 나온 "파라미터 효율적 적응" 방법들이 이미 존재하는데, 왜 그걸로는 부족해서 LoRA가 필요한지를 설명하는 파트

크게 두 갈래의 기존 접근을 검토 $\rightarrow$ **Adapter layers** 추가 방식과 **입력 활성화(prompt) 최적화** 방식

2) Adapter Layers의 문제: Inference Latency 증가

- Adapter: Transformer 블록 사이사이에 작은 레이어를 끼워 넣는 방식
- 두 가지 설계를 비교 대상으로 삼음 $\rightarrow$ Houlby et al.의 원조 설계(블록당 adapter 2개), Lin et al.의 개선 설계(블록당 1개 + LayerNorm 추가)

- 문제: 이 adapter 레이어들이 **순차적으로(sequentially)** 처리돼야 한다는 점
 - 대형 신경망은 하드웨어 병렬성(parallelism)을 활용해 latency를 낮추는데, adapter는 이 병렬화 흐름에 끼어들어가 추가 연산을 강제함
 - Adapter 파라미터 수 자체는 적어도(전체의 <1%), 순차 처리라는 구조적 특성 때문에 지연이 발생

3) 실측 근거 (Table 1)

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
Adapter ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
Adapter ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. “ $|\Theta|$ ” denotes the number of trainable parameters in adapter layers. Adapter^L and Adapter^H are two variants of adapter tuning, which we describe in [Section 5.1](#). The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in [Appendix B](#).

GPT-2 medium에서 batch size가 작을수록(특히 online inference처럼 batch size=1인 경우) latency 증가가 두드러짐

batch size 1, seq len 128 조건에서 AdapterH는 파라미터가 11M밖에 안 되는데도 latency가

+20.7%~+30.3% 증가

모델을 여러 GPU로 shard(분산)하는 경우엔 문제가 더 심해짐 → adapter의 추가 depth가 AllReduce, Broadcast 같은 GPU 간 동기화 연산을 더 필요로 하기 때문

4) Prompt(Prefix) 최적화 방식의 문제: 최적화 난이도

Prefix-tuning(Li & Liang)으로 대표되는 접근 → 입력 시퀀스 앞에 학습 가능한 토큰(prefix)을 붙여서, 그 prefix의 임베딩만 학습하는 방식

저자들이 관찰한 문제 두 가지:

1. 최적화가 어려움 → 학습이 잘 수렴하지 않는 경향
2. 학습 파라미터 수를 늘려도 성능이 **단조증가(monotonic)**하지 않음
 - 오히려 특정 지점 이후 성능이 떨어지는 non-monotonic 패턴

더 근본적인 문제 → prefix가 시퀀스 길이의 일부를 차지해버리기 때문에, 실제 다운스트림 태스크 처리에 쓸 수 있는 시퀀스 길이가 줄어들

저자들은 이것이 prompt tuning류 방법이 다른 방법보다 성능이 떨어지는 이유일 거라고 추정

5) 정리

두 갈래 접근 모두 "효율성 vs 모델 품질" 사이에서 trade-off 갖는다는 게 이 섹션의 결론

- Adapter → 파라미터는 적지만 **inference latency 증가**
- Prefix-tuning → latency는 없지만 **최적화 어려움 + 시퀀스 길이 손해**

이 두 가지 한계를 모두 피해가는 방법이 필요하다는 문제의식이 바로 다음 섹션(Section 4. Method)으로 이어지는 동기가 됨 → **inference latency도 없고, 시퀀스 길이도 안 깎아먹는** 방법을 만들겠다는 게 LoRA의 설계 목표

4. Our Method

원칙적으로는 딥러닝 모델의 어떤 dense layer에도 적용 가능한 방법이지만, 논문에서는 Transformer 언어모델의 특정 가중치에만 초점을 맞춰 실험 진행

4.1 LOW-RANK-PARAMETRIZED UPDATE MATRICES

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3)$$

1) 기본 관찰

신경망은 행렬곱을 수행하는 dense layer를 다수 포함하며, 이 가중치 행렬들은 일반적으로 full-rank
Aghajanyan et al. (2020)의 발견 → 사전학습 언어모델은 낮은 "intrinsic dimension"을 가지며, 더 작은 부분
공간(subspace)으로 random projection해도 여전히 효율적으로 학습 가능

저자들의 가설 → 적응(adaptation) 과정에서 가중치의 변화량 역시 낮은 "intrinsic rank"를 가질 것

2) 저랭크 분해로 업데이트 제약

사전학습 가중치 행렬 $W_0 \in \mathbb{R}^{d \times k}$ 가 있을 때, 그 업데이트를 다음과 같이 저랭크 분해로 표현

$$W_0 + \Delta W = W_0 + BA$$

- $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$
- $r \ll \min(d, k)$

학습 중 W_0 은 고정되어 gradient를 받지 않음, A 와 B 만 학습 가능한 파라미터 포함

W_0 과 $\Delta W = BA$ 는 동일한 입력을 받아 각각 곱해진 후, 출력 벡터를 좌표별로(coordinate-wise) 합산

학습 중 W_0 은 고정되어 gradient를 받지 않음, A 와 B 만 학습 가능한 파라미터 포함

W_0 과 $\Delta W = BA$ 는 동일한 입력을 받아 각각 곱해진 후, 출력 벡터를 좌표별로(coordinate-wise) 합산

3) Forward pass 수식 (Eq. 3)

$h = W_0x$ 였던 기존 forward pass가 다음과 같이 수정됨

$$h = W_0x + \Delta Wx = W_0x + BAx$$

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3)$$

4) 초기화와 스케일링

- A : 랜덤 가우시안 분포로 초기화
- B : 0으로 초기화
- 결과적으로 학습 시작 시점엔 $\Delta W = BA = 0$

ΔWx 에 α/r 을 곱해 스케일링 (α 는 r 에 대한 상수)

Adam으로 최적화할 때 α 를 튜닝하는 것은 초기화를 적절히 스케일한다면 학습률(learning rate)을 튜닝하는 것과 거의 동일한 효과

그래서 저자들은 처음 시도하는 r 값으로 α 를 단순히 고정하고 별도로 튜닝하지 않음

이 스케일링은 r 을 바꿔가며 실험할 때 하이퍼파라미터를 재튜닝해야 할 필요성을 줄여주는 역할 (Yang & Hu, 2021)

5) Full fine-tuning의 일반화로서의 LoRA

더 일반화된 형태의 파인튜닝은 사전학습 파라미터 중 일부 subset만 학습하는 것을 허용

LoRA는 한 걸음 더 나아가, 가중치 행렬에 대한 누적 gradient 업데이트가 적응 과정에서 반드시 full-rank일 필요가 없다고 봄

의미 → 모든 가중치 행렬에 LoRA를 적용하고 모든 bias까지 학습한다면, LoRA의 rank r 을 사전학습 가중치 행렬의 rank만큼 크게 설정함으로써 full fine-tuning의 표현력(expressiveness)을 거의 그대로 회복 가능
즉 학습 가능한 파라미터 수를 늘려갈수록, LoRA는 원본 모델을 학습하는 것에 점점 가까워지는 반면 adapter 기반 방법은 결국 MLP로 수렴하고, prefix 기반 방법은 긴 입력 시퀀스를 처리할 수 없는 모델로 수렴

6) 추가 Inference Latency 없음

배포 시 $W = W_0 + BA$ 를 명시적으로 계산해 저장한 뒤, 평소처럼 추론 수행 가능

다른 다운스트림 태스크로 전환할 때는 BA 를 빼고 다른 $B'A'$ 를 더하면 됨 → 메모리 오버헤드가 매우 적은 빠른 연산

이 덕분에 배포된 파인튜닝 모델 대비 구조적으로 추가 추론 지연이 전혀 발생하지 않음이 보장됨

4.2 APPLYING LORA TO TRANSFORMER

1) 적용 범위

원칙적으로 신경망의 어떤 가중치 행렬 subset에도 LoRA 적용 가능해 학습 파라미터 수를 줄일 수 있음

Transformer 아키텍처에는 self-attention 모듈에 4개(W_q, W_k, W_v, W_o), MLP 모듈에 2개의 가중치 행렬 존재

W_q (혹은 W_k, W_v)는 출력 차원이 보통 attention head 단위로 나뉘어 있지만, 논문에서는 $d_{model} \times d_{model}$ 크기의 단일 행렬로 취급

이 연구에서는 단순성과 파라미터 효율성을 위해 **attention 가중치만 적응시키고 MLP 모듈은 고정**(다운스트림 태스크에서 학습 안 함)

Transformer 내 서로 다른 유형의 attention 가중치 행렬을 적응시키는 효과는 Section 7.1에서 추가로 다룬 MLP layer, LayerNorm layer, bias에 대한 적응 실험은 future work로 남김

2) 실질적 이점: 메모리 및 저장공간 절감

가장 큰 이점은 메모리/저장 사용량 감소

Adam으로 학습되는 대형 Transformer의 경우, $r \ll d_{model}$ 이면 고정된 파라미터에 대한 옵티마이저 상태를 저장할 필요가 없어 VRAM 사용량을 최대 2/3까지 절감

GPT-3 175B 기준 → 학습 중 VRAM 소비가 1.2TB에서 350GB로 감소

$r = 4$ 로 query, value projection 행렬만 적응시킬 경우 → 체크포인트 크기가 약 10,000배 감소 (350GB → 35MB)

이를 통해 훨씬 적은 GPU로 학습 가능하고 I/O 병목현상 회피 가능

또 다른 이점 → 배포된 상태에서 전체 파라미터가 아닌 LoRA 가중치만 교체하면 되므로 훨씬 낮은 비용으로 태스크 전환 가능

이는 사전학습 가중치를 VRAM에 올려둔 머신에서 여러 커스터마이징된 모델을 실시간으로 교체 투입 가능하게 함
GPT-3 175B 기준 full fine-tuning 대비 학습 중 **25% 속도 향상**도 관찰됨 → 대다수 파라미터에 대한 gradient 계산이 불필요하기 때문

3) 한계점

A 와 B 를 W 에 흡수시켜 추가 추론 지연을 없애기로 선택한 경우, 서로 다른 태스크의 입력을 하나의 배치(batch)에 섞어서 처리하는 것이 간단하지 않음

다만 latency가 중요하지 않은 시나리오라면, 가중치를 병합하지 않고 배치 내 샘플마다 동적으로 사용할 LoRA 모듈을 선택하는 것도 가능

5. Empirical Experiments

RoBERTa, DeBERTa, GPT-2에서 LoRA의 다운스트림 태스크 성능을 평가한 뒤, GPT-3 175B로 확장 NLU(자연어이해)부터 NLG(자연어생성)까지 폭넓은 태스크 커버

RoBERTa, DeBERTa엔 GLUE 벤치마크 사용

GPT-2엔 Li & Liang (2021) 세팅을 따라 직접 비교하고, WikiSQL(NL → SQL), SAMSum(대화 요약)을 대규모 GPT-3 실험에 추가

모든 실험엔 NVIDIA Tesla V100 사용

5.1 Baselines

가능한 경우 기존 연구의 세팅을 그대로 재현하고 보고된 수치를 재사용 → 그래서 일부 baseline은 특정 실험에만 등장

1. Fine-Tuning (FT)

- 가장 흔한 적응 방식
- 모델을 사전학습 가중치/bias로 초기화한 뒤 모든 파라미터가 gradient 업데이트를 받음
- 변형: 일부 레이어만 업데이트하고 나머지는 고정 → GPT-2에서 마지막 두 레이어만 적응시키는 FTTop2를 baseline으로 포함

2. Bias-only / BitFit

- bias 벡터만 학습하고 나머지는 전부 고정
- BitFit(Zaken et al., 2021)에서 다뤄진 baseline과 동일한 개념

3. Prefix-embedding tuning (PreEmbed)

- 입력 토큰 사이에 특수 토큰을 삽입, 이 토큰들은 학습 가능한 word embedding을 가짐 (모델 vocabulary엔 없는 토큰)
- "prefixing"(프롬프트 앞에 붙임)과 "infixing"(프롬프트 뒤에 붙임) 두 방식 존재
- l_p : prefix 토큰 수
 l_i : infix 토큰 수
- 학습 파라미터 수 $|\Theta| = d_{model} \times (l_p + l_i)$

4. Prefix-layer tuning (PreLayer)

Prefix-embedding tuning의 확장판

특수 토큰의 word embedding만 학습하는 게 아니라, 모든 Transformer layer 이후의 activation을 학습
이전 레이어에서 계산된 activation을 학습 가능한 값으로 그냥 대체

학습 파라미터 수 $|\Theta| = L \times d_{model} \times (l_p + l_i)$ ($L = \text{Transformer layer 수}$)

5. Adapter tuning

self-attention 모듈(및 MLP 모듈)과 이후의 residual connection 사이에 adapter layer 삽입

- $Adapter^H$: Houlsby et al. (2019)의 원조 설계, 블록당 fully-connected layer 2개 + 비선형 함수
- $Adapter^L$: Lin et al. (2020)의 설계, MLP 모듈 이후 + LayerNorm 이후에만 적용
- $Adapter^P$: Pfeiffer et al. (2021)의 유사 설계
- $Adapter^D$: AdapterDrop(Ruckle et al., 2020), 효율성을 위해 일부 adapter layer를 드롭
파라미터 수 =

$$|\Theta| = \hat{L}_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$$

- $\hat{L}_{Adpt}$: 어댑터 층의 수
- $\hat{L}_{LN}$: 학습 가능한 LayerNorm의 수(예: AdapterL의 경우)

6. LoRA

- 기존 가중치 행렬에 병렬로 학습 가능한 rank decomposition 행렬 쌍 추가
- 대부분의 실험에서 단순성을 위해 W_q, W_v 에만 적용
- 파라미터 수 =

$$|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r,$$

- $\hat{L}_{LoRA}$: LoRA를 적용하는 가중치 행렬의 수

5.2 RoBERTa - Base/Large

- RoBERTa는 BERT의 사전학습 레시피를 최적화, 파라미터 수를 크게 늘리지 않고도 성능 향상
- GLUE 리더보드에서 최근엔 더 큰 모델들에 밀렸지만, 크기 대비 여전히 경쟁력 있고 실무자들 사이에서 인기 있는 모델
- HuggingFace Transformers 라이브러리의 RoBERTa base(125M), RoBERTa large(355M) 사용
- GLUE 벤치마크 태스크에서 여러 효율적 적응 방식 성능 비교
- Houlsby et al. (2019), Pfeiffer et al. (2021)의 세팅도 함께 재현

공정한 비교를 위한 두 가지 조정

1. 모든 태스크에 동일한 batch size, seq len 128 사용 (adapter baseline과 맞춤)
2. MRPC, RTE, STS-B 태스크는 fine-tuning baseline처럼 MNLI로 이미 적응된 모델이 아니라, 사전학습 모델 자체에서 초기화

이 제한된 세팅을 따르는 실험 결과엔 + 표시

결과는 Table 2 상단 세 섹션에 제시, 하이퍼파라미터는 Section D.1 참고

Model & Method	# Trainable Parameters									
		MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^L)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1 $\pm$ 2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3 $\pm$ 3	90.8 $\pm$ 1	86.6 $\pm$ 7	91.5 $\pm$ 2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6 $\pm$ 2	96.2 $\pm$ 5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$ 3	91.6 $\pm$ 1	87.4 $\pm$ 2.5	92.6 $\pm$ 2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3 $\pm$ 1.0	94.8 $\pm$ 2	91.9 $\pm$ 1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5 $\pm$ 3	96.6 $\pm$ 2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8 $\pm$ 3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6 $\pm$ 2	96.2 $\pm$ 5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$ 3	91.6 $\pm$ 2	85.2 $\pm$ 1.1	92.3 $\pm$ 5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9 $\pm$ 2	96.9 $\pm$ 2	92.6 $\pm$ 6	72.4 $\pm$ 1.1	96.0 $\pm$ 1	92.9 $\pm$ 1	94.9 $\pm$ 4	93.0 $\pm$ 2	91.3

Table 2: RoBERTa_{base}, RoBERTa_{large}, and DeBERTa_{XXL} with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew’s correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to [Houlsby et al. \(2019\)](#) for a fair comparison.

5.3 DeBERTa XXL

DeBERTa(He et al., 2021)는 BERT의 최신 변형, 훨씬 큰 스케일로 학습되어 GLUE, SuperGLUE 같은 벤치마크에서 매우 경쟁력 있는 성능

LoRA가 완전히 fine-tuning된 DeBERTa XXL(1.5B)의 GLUE 성능을 여전히 따라잡을 수 있는지 평가

결과는 Table 2 하단 섹션에 제시, 하이퍼파라미터는 Section D.2 참고

5.4 GPT-2 - Medium/Large

RoBERTa/DeBERTa로 LoRA가 NLU에서 fine-tuning의 경쟁력 있는 대안임을 보였으니, GPT-2 medium/large 같은 NLG 모델에서도 통하는지 확인

직접 비교를 위해 Li & Liang (2021)의 세팅을 최대한 그대로 유지

지면 제약으로 본문엔 E2E NLG Challenge 결과(Table 3)만 제시, WebNLG/DART 결과는 Section F.1 참고

사용된 하이퍼파라미터는 Section D.3에 목록화

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$ 6	8.50 $\pm$ 0.7	46.0 $\pm$ 2	70.7 $\pm$ 2	2.44 $\pm$ 0.1
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$ 1	8.85 $\pm$ 0.2	46.8 $\pm$ 2	71.8 $\pm$ 1	2.53 $\pm$ 0.2
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$ 1	8.68 $\pm$ 0.3	46.3 $\pm$ 0	71.4 $\pm$ 2	2.49 $\pm$ 0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$ 3	8.70 $\pm$ 0.4	46.1 $\pm$ 1	71.3 $\pm$ 2	2.45 $\pm$ 0.2
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$ 1	8.89 $\pm$ 0.2	46.8 $\pm$ 2	72.0 $\pm$ 2	2.47 $\pm$ 0.2

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

5.5 Scaling up to GPT-3 175B

LoRA에 대한 최종 스트레스 테스트로 1,750억 파라미터의 GPT-3까지 확장

학습 비용이 매우 높아, 모든 항목에 대해 표준편차를 제공하는 대신 특정 태스크에 대한 typical 표준편차만 보고 하이퍼파라미터는 Section D.4 참고

주요 결과 (Table 4)

- LoRA가 세 데이터셋(WikiSQL, MNLI-matched, SAMSum) 모두에서 fine-tuning baseline과 비슷하거나 이를 능가

주목할 발견: 파라미터를 늘린다고 항상 성능이 좋아지진 않음 (Figure 2)

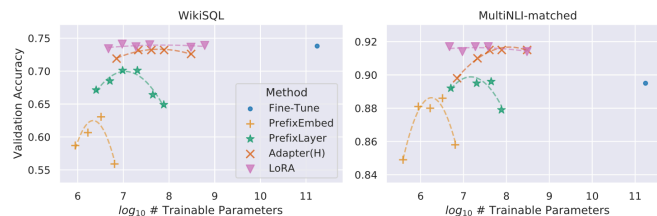


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See [Section F.2](#) for more details on the plotted data points.

- prefix-embedding tuning: 특수 토큰이 256개를 넘으면 성능 크게 하락
- prefix-layer tuning: 특수 토큰이 32개를 넘으면 성능 크게 하락
- Li & Liang (2021)에서도 유사한 관찰 존재
- 저자들의 추정 → 특수 토큰이 많아질수록 입력 분포가 사전학습 데이터 분포에서 점점 멀어지기 때문일 것 (단, 이 현상에 대한 심층 조사는 본 논문 범위 밖)

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

6. Related Works

6.1 Transformer Language Models

Transformer(Vaswani et al., 2017)는 self-attention을 적극 활용하는 sequence-to-sequence 아키텍처 Radford et al. (a)이 이를 Transformer decoder 스택을 사용한 autoregressive 언어모델링에 적용 이후 Transformer 기반 언어모델이 NLP를 지배, 다수 태스크에서 SOTA 달성

BERT(Devlin et al., 2019b), GPT-2(Radford et al., b) 등장과 함께 새로운 패러다임 부상

공통점 → 일반 도메인 데이터로 사전학습한 뒤 태스크별 데이터로 fine-tuning하는 방식이, 태스크별 데이터로 직접 학습하는 것보다 훨씬 큰 성능 향상 제공

Transformer를 더 크게 학습할수록 일반적으로 더 나은 성능 → 여전히 활발한 연구 방향

GPT-3(Brown et al., 2020)는 1,750억 파라미터로 지금까지 학습된 단일 Transformer 언어모델 중 최대 규모

6.2 Prompt Engineering and Fine-Tuning

GPT-3 175B는 소수의 추가 학습 예시만으로 행동을 조정할 수 있지만, 그 결과가 입력 프롬프트에 크게 좌우됨 (Brown et al., 2020)

이 때문에 모델 성능을 극대화하기 위해 프롬프트를 구성/포맷팅하는 경험적 기술이 필요 → prompt engineering(또는 prompt hacking)이라 불림

Fine-tuning은 일반 도메인에서 사전학습된 모델을 특정 태스크에 맞게 재학습하는 방식 (Devlin et al., 2019b; Radford et al., a)

변형 → 파라미터의 일부 subset만 학습하는 방식도 존재 (Devlin et al., 2019b; Collobert & Weston, 2008)
다만 실무자들은 다운스트림 성능을 극대화하기 위해 흔히 파라미터 전체를 재학습

문제 → GPT-3 175B의 규모 때문에 일반적인 방식의 fine-tuning 수행이 어려움

- 생성되는 체크포인트 크기가 매우 큼
- 사전학습과 동일한 메모리 풋프린트가 필요해 하드웨어 진입 장벽이 높음

6.3 Parameter-Efficient Adaptation

기존 신경망의 레이어 사이에 adapter layer를 삽입하는 방식이 다수 제안됨 (Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020)

LoRA도 유사한 bottleneck 구조를 사용해 가중치 업데이트에 저랭크 제약을 부과한다는 점에서 관련성 있음

핵심 기능적 차이점 → LoRA로 학습된 가중치는 추론 시 메인 가중치와 병합 가능해 latency를 유발하지 않음, 반면 adapter layer는 그렇지 않음 (Section 3에서 다룸)

동시대의 확장 연구 → COMPACTER (Mahabadi et al., 2021)

- adapter layer를 사전 정의된 weight sharing scheme과 함께 Kronecker product로 파라미터화하는 방식
- LoRA를 다른 텐서곱 기반 방법과 결합하면 파라미터 효율성을 더 높일 수 있을 것으로 보이나, 이는 future work로 남김

최근 흐름 → fine-tuning 대신 입력 word embedding을 최적화하는 방식 제안 다수 (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021)

- prompt engineering을 연속적이고 미분 가능하게 일반화한 개념으로 볼 수 있음
- 실험 섹션에서 Li & Liang (2021)과 비교 결과 포함
- 한계 → 프롬프트에 더 많은 특수 토큰을 사용하는 방식으로만 스케일업 가능 → positional embedding이 학습되는 경우, 이 토큰들이 태스크용 토큰이 쓸 시퀀스 길이를 잠식

6.4 Low-Rank Structures in Deep Learning

저랭크 구조(low-rank structure)는 머신러닝에서 매우 흔한 현상

다수의 머신러닝 문제가 특정한 intrinsic low-rank 구조를 가짐 (Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013)

특히 과매개변수화(over-parametrized)된 신경망을 사용하는 딥러닝 태스크 다수에서, 학습된 신경망이 학습 후 저랭크 특성을 갖는다는 것이 알려져 있음 (Oymak et al., 2019)

일부 선행 연구는 원본 신경망을 학습할 때 저랭크 제약을 명시적으로 부과하기도 함 (Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Khodak et al., 2021; Denil et al., 2014)

저자들이 강조하는 차별점 → 위 선행 연구들 중 어느 것도, 다운스트림 태스크 적응을 위해 **고정된(frozen) 모델에 저랭크 업데이트를 적용**하는 방식은 다루지 않았다는 점

이론적 문헌에서의 근거

- 신경망은 특정 저랭크 구조를 가진 개념 클래스(concept class)에서, 대응하는 (finite-width) neural tangent kernel을 포함한 다른 고전적 학습 방법들보다 뛰어난 성능을 보인다고 알려짐(Allen-Zhu et al., 2019; Li & Liang, 2018; Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a)
- Allen-Zhu & Li (2020b)의 또 다른 이론적 결과 → 저랭크 적응(low-rank adaptation)이 adversarial training에 유용할 수 있음을 시사

정리 → 이러한 문헌들을 종합해볼 때, 저자들이 제안하는 저랭크 적응 업데이트는 기존 연구들에 의해 충분히 동기 부여된 접근이라고 판단

7. UNDERSTANDING THE LOW-RANK UPDATES

LoRA의 경험적 이점을 확인했으니, 다운스트림 태스크에서 학습된 저랭크 적응의 속성을 더 깊이 설명하고자 함
저랭크 구조의 의미 → 하드웨어 진입 장벽을 낮춰 여러 실험을 병렬로 돌릴 수 있게 해줄 뿐 아니라, 업데이트 가중치가 사전학습 가중치와 어떻게 연관되는지에 대한 해석 가능성(interpretability)도 제공

GPT-3 175B에 연구 초점을 맞춤 → 태스크 성능에 악영향 없이 학습 파라미터를 최대 10,000배까지 줄인 사례이기 때문

세 가지 질문에 답하고자 실험 진행

1. 파라미터 예산이 제한된 상황에서, 다운스트림 성능을 극대화하려면 사전학습 Transformer의 어떤 가중치 행렬 subset을 적응시켜야 하는가?
2. "최적의" 적응 행렬 ΔW 는 실제로 rank-deficient(랭크 부족)한가? 그렇다면 실무에서 어느 정도의 rank를 쓰는 게 적절한가?
3. ΔW 와 W 사이의 관계는 무엇인가? ΔW 가 W 와 높은 상관관계를 갖는가? ΔW 는 W 에 비해 얼마나 큰가?

질문 (2), (3)에 대한 답이 사전학습 언어모델을 다운스트림 태스크에 활용하는 근본 원리를 밝혀줄 것으로 기대

7.1 WHICH WEIGHT MATRICES IN TRANSFORMER SHOULD WE APPLY LORA TO?

1) 실험 세팅

제한된 파라미터 예산 하에서, 어떤 유형의 가중치를 LoRA로 적응시켜야 최상의 다운스트림 성능을 얻는지 확인
Section 4.2에서 언급했듯 self-attention 모듈의 가중치 행렬만 고려
GPT-3 175B에 파라미터 예산 18M(FP16 저장 시 약 35MB) 설정

- 한 종류의 attention 가중치만 적응 → $r = 8$
- 두 종류를 적응 → $r = 4$
- 96개 레이어 전체에 적용

2) 결과 (Table 5)

Weight Type Rank r	# of Trainable Parameters = 18M						
	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

- W_q 또는 W_k 하나에만 전체 파라미터를 몰아준 경우 성능이 눈에 띄게 낮음
- W_q 와 W_v 둘 다를 적응시킨 경우가 전체적으로 최상의 결과

3) 해석

$r=4$ 처럼 작은 rank라도 ΔW 의 정보를 충분히 담아낼 수 있음을 시사

즉 단일 유형의 가중치에 더 큰 rank를 부여하는 것보다, 더 많은 종류의 가중치 행렬을 (작은 rank로라도) 함께 적응시키는 편이 더 바람직함

7.2 WHAT IS THE OPTIMAL RANK r FOR LORA?

1) 실험 세팅

rank r 이 모델 성능에 미치는 영향에 주목

비교 대상 $\rightarrow W_q, W_v, W_q, W_k, W_v, W_o$, 그리고 W_q 단독 적응

2) 결과 (Table 6)

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r . To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r . We conduct a similar experiment on GPT-2 in [Section H.2](#)

놀랍게도 매우 작은 r 로도 LoRA가 경쟁력 있는 성능을 보임 (W_q 단독보다 W_q, W_v 조합에서 더 두드러짐)

이는 업데이트 행렬 ΔW 가 매우 작은 "intrinsic rank"를 가질 수 있음을 시사

3) 추가 검증

이 발견을 뒷받침하기 위해, 서로 다른 r 선택 및 서로 다른 random seed로 학습된 결과들이 학습한 subspace 간 중첩(overlap) 정도를 확인

결론 $\rightarrow r$ 을 늘린다고 해서 더 의미 있는 subspace를 커버하는 게 아님 $\rightarrow$ 저랭크 적응 행렬만으로 충분하다는 근거

4) 단서

모든 태스크나 데이터셋에 작은 r 이 통할 거라고 기대해선 안 됨

사고 실험 $\rightarrow$ 만약 다운스트림 태스크가 사전학습에 쓰인 언어와 완전히 다른 언어라면, 모델 전체를 재학습하는 것 (즉 $r = d_{model}$ 인 LoRA와 유사)이 작은 r 의 LoRA보다 확실히 더 나은 성능을 낼 수 있음

7.2.1 Subspace similarity between different r

1. 실험 목적

같은 사전학습 모델에서 학습된 $A_{r=8}$ 과 $A_{r=64}$ (rank 8, 64로 학습된 적응 행렬)를 놓고, singular value decomposition을 수행해 right-singular unitary 행렬 $U_{A_{r=64}}, U_{A_{r=8}}$ 를 얻음
 질문 → $U_{A_{r=8}}$ 의 top i 개 singular vector가 펼치는 subspace가, $U_{A_{r=64}}$ 의 top j 개 singular vector가 펼치는 subspace에 얼마나 포함되는가

▼ 기호 정리

기호	의미
W_0	사전학습된 가중치 행렬 (frozen, 학습 안 됨)
ΔW	W_0 에 더해지는 변화량 LoRA에서는 $\Delta W = BA$ 로 저랭크 분해
B, A	ΔW 를 만들어내는 두 개의 저랭크 행렬. 이 둘만 학습됨
W	최종 가중치 = $W_0 + \Delta W$ (배포 시 병합해서 쓰는 행렬)

$$W_0(\text{고정}) + \underbrace{BA}_{\Delta W(\text{학습 대상})} = W(\text{최종 가중치})$$

2. 측정 방법 (Eq. 4)

Grassmann distance 기반의 normalized subspace similarity로 측정

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^i U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1]$$

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

$\phi(\cdot)$ 의 범위는 $[0, 1] \rightarrow 1$ 이면 subspace가 완전히 겹침, 0이면 완전히 분리

지면 제약으로 96개 레이어 중 48번째 레이어만 살펴봄(다른 레이어에서도 결론은 동일, Section H.1 참고)

3. 핵심 관찰 (Figure 3)

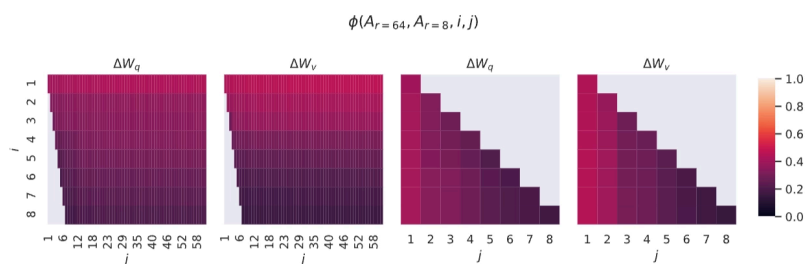


Figure 3: Subspace similarity between column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v . The third and the fourth figures zoom in on the lower-left triangle in the first two figures. The top directions in $r=8$ are included in $r=64$, and vice versa.

top singular vector에 대응하는 방향은 $A_{r=8}$ 과 $A_{r=64}$ 사이에 상당히 겹치지만, 나머지 방향들은 그렇지 않음
 구체적으로 → ΔW_v (또는 ΔW_q)의 $A_{r=8}$ 과 $A_{r=64}$ 가 normalized similarity 0.5 이상으로 겹치는 1차원 subspace를 공유

이는 GPT-3의 다운스트림 태스크에서 왜 $r=1$ 만으로도 꽤 좋은 성능이 나오는지에 대한 설명이 됨

4. 해석

$A_{r=8}$ 과 $A_{r=64}$ 는 동일한 사전학습 모델로 학습됐으므로, top singular-vector 방향들이 가장 유용한 정보를 담고 있고, 나머지 방향들은 학습 중 누적된 무작위 노이즈에 가까울 가능성이 큼
따라서 적응 행렬(adaptation matrix)은 실제로 매우 낮은 rank를 가질 수 있음

7.2.2 Subspace similarity between different random seeds

1) 실험 목적

위 발견을 다른 각도에서 재확인 → $r=64$ 로 학습하되 random seed만 다르게 한 두 번의 실행(run) 사이의 normalized subspace similarity를 측정 (Figure 4)

2) 결과

ΔW_q 가 ΔW_v 보다 더 높은 "intrinsic rank"를 갖는 것으로 나타남

근거 → ΔW_q 의 경우 두 실행이 더 많은 공통 singular value 방향을 학습함 → Table 6에서 관찰된 경험적 결과와 일치

3) 대조군

비교를 위해 서로 완전히 무관한 두 개의 random Gaussian 행렬도 함께 plot

이 둘은 서로 어떤 singular value 방향도 공유하지 않음 → LoRA로 학습된 행렬들이 우연이 아니라 실제로 의미 있는 방향을 공유한다는 것을 대비시켜 보여줌

7.3 HOW DOES THE ADAPTATION MATRIX ΔW COMPARE TO W ?

1) 연구 질문

ΔW 와 W 의 관계를 조사

- ΔW 가 W 와 높은 상관관계를 갖는가? (수학적으로: ΔW 가 W 의 top singular direction에 주로 포함되어 있는가?)
- ΔW 의 "크기"는 W 의 대응 방향과 비교했을 때 어느 정도인가?

2) 측정 방법

$U^\top W V^\top$ 을 계산해 W 를 ΔW 의 r 차원 subspace로 projection

- U/V : ΔW 의 left/right singular-vector 행렬
- $\|U^\top W V^\top\|_F$ 와 $\|W\|_F$ 의 Frobenius norm을 비교
- 비교 대상으로 U, V 대신 W 자체의 top r singular vector를 사용한 경우, 그리고 random 행렬을 사용한 경우도 함께 계산

3) 결과 (Table 7)

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

Table 7: The Frobenius norm of $U^\top W_q V^\top$ where U and V are the left/right top r singular vector directions of either (1) ΔW_q , (2) W_q , or (3) a random matrix. The weight matrices are taken from the 48th layer of GPT-3.

GPT-3의 48번째 레이어 가중치를 대상으로 측정한 결과 세 가지 결론 도출

1. ΔW 는 random 행렬보다 W 와 훨씬 강한 상관관계를 가짐 → ΔW 가 이미 W 에 존재하는 일부 feature를 담고 있다는 뜻
2. ΔW 는 W 의 top singular 방향을 단순히 반복하는 게 아니라, **W 에서 강조되지 않았던 방향을 증폭시킴**
3. 증폭 계수(amplification factor)가 상당히 큼 → $r=4$ 일 때 약 $21.5 \approx 6.91/0.32$ 배 ($r=64$ 일 때 증폭 계수가 더 작은 이유는 Section H.4 참고)

4) 종합 해석

이 결과는 저랭크 적응 행렬이 **일반 사전학습 모델에서는 학습됐지만 크게 강조되지 않았던, 특정 다운스트림 태스크에 중요한 feature들을 증폭시키는** 역할을 한다는 것을 시사

Section H.3에서는 W_q 의 top singular 방향을 더 많이 포함시킬수록 이 상관관계가 어떻게 변하는지에 대한 시각화도 추가로 제공

8. CONCLUSION AND FUTURE WORK

1) 결론

- 거대 언어모델의 fine-tuning은 필요한 하드웨어 비용과, 태스크별 독립 인스턴스를 호스팅하는 데 드는 저장/전환 비용 측면에서 지나치게 비쌈
- LoRA 제안 → 추론 지연(inference latency)을 유발하지 않고, 입력 시퀀스 길이도 줄이지 않으면서, 높은 모델 품질을 유지하는 효율적인 적응 전략
- 대부분의 모델 파라미터를 공유함으로써, 서비스로 배포될 때 빠른 태스크 전환도 가능
- 이 연구는 Transformer 언어모델에 초점을 맞췄지만, 제안된 원리는 dense layer를 가진 어떤 신경망에도 일반적으로 적용 가능

2) 향후 연구 방향

1. LoRA를 다른 효율적 적응 기법들과 결합 → 상호 보완적인(orthogonal) 성능 향상 가능성
2. Fine-tuning 또는 LoRA의 작동 메커니즘 자체가 아직 명확하지 않음 → 사전학습 중 학습된 feature들이 다운스트림 태스크에 잘 맞도록 어떻게 변환되는지는 여전히 미해결 문제. 저자들은 LoRA가 이 질문에 답하기에 full fine-tuning보다 더 다루기 쉬운(tractable) 틀을 제공한다고 봄
3. 현재는 어떤 가중치 행렬에 LoRA를 적용할지 대부분 휴리스틱에 의존 → 더 원칙적인 선택 방법이 있는가?
4. ΔW 가 rank-deficient하다는 발견은, W 자체도 rank-deficient할 수 있음을 시사 → 향후 연구의 영감이 될 수 있는 지점